

SDD Brownfield

Lección 2 – Spec-Driven Development

Cambiar un codebase que ya existe sin perder el control.

Qué vamos a ver hoy

1 Spot review – un equipo sorteado presenta la tarea de la Lección 1

2 La realidad brownfield – por qué "refactorizá X" falla

3 Descubrimiento antes de especificar – mapear antes de tocar

4 Acotar specs brownfield – angostas, explícitas, con fuera-de-alcance

5 Flujo SDD brownfield – capsules, orden, regresión

6 Briefing de la tarea – tmux con SSH nativo (en equipo)

Al final de esta clase podés...

Contrastar greenfield y brownfield – y por qué brownfield exige descubrimiento

Evaluar la spec, el plan y la implementación de otro equipo contra criterios SDD

Explorar un codebase desconocido con agentes antes de escribir una spec

Acotar cambios con precisión para que el agente no driftee

Congelar lo entregado en una context capsule para la próxima spec

Spot review

Tarea de la Lección 1 – un par, en vivo

Qué estamos mirando

Un **equipo sorteado** nos muestra su entregable de **gcsgrep**.



Calidad de la spec

Propósito y alcance claros, FRs atómicos, preguntas abiertas resueltas, umbrales reales en los NFRs.



Cobertura de VCs

Cada FR y BR tiene un chequeo observable. Caminos de falla, no solo el happy path.



Base → build

Qué tan bien el base context refinado atravesó Especificar → Verificar.

Esto es un **ensayo de revisión** – el mismo ojo crítico que hoy vamos a aplicar a código brownfield.

La realidad brownfield

El código ya existe – y tiene opiniones

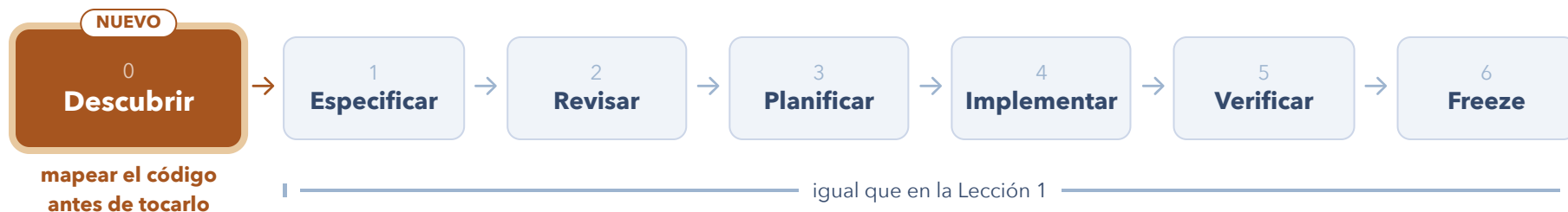
Greenfield vs. brownfield

	Greenfield	Brownfield
Punto de partida	hoja en blanco	sistema existente
Restricciones	las que elegís	convenciones ya horneadas
Riesgo principal	ambigüedad	ambigüedad + regresión
Primer trabajo	refinar la idea	entender qué hay
Éxito	hace lo que especificamos	...y no rompió nada más

Mismo pipeline. Un paso nuevo adelante: **descubrimiento**.

Mismo pipeline, un paso nuevo

Los seis pasos de la Lección 1 no cambian. Brownfield les agrega uno **adelante**.



Entrada

el repo y el cambio pedido

Modo

solo lectura: el agente lee, busca y traza, pero no edita

Salida

notas de exploración: la entrada de Especificar

En greenfield, Especificar arranca del **base context**. En brownfield, de lo que **Descubrir** encontró en el código.

"Che, refactorizá X"

Apuntá un agente a un repo maduro con una instrucción vaga, y con toda confianza va a...

Pasar por alto una convención implícita

Romper un caller que nunca leyó

Reescribir más de lo que pediste

"Mejorar" código no relacionado

Duplicar un helper que ya existía

Regresionar un test que no corrió

La context window contiene una **fracción** del sistema. Todo lo que el agente no ve, lo *adivina* – y en código existente, adivinar es una regresión.

La ambigüedad en greenfield entrega un feature equivocado. En brownfield **rompe uno que funcionaba**.

El problema del contexto oculto

Lo que falta: está en el sistema, pero no está escrito



Arquitectura existente

Capas, límites y patrones a los que te adaptás, no que peleás.



Convenciones implícitas

Nombres, errores, estilo de tests: acordados, nunca escritos.



Documentación faltante

El porqué vive en commits, tickets y personas, no en el README.



Deuda técnica

Workarounds que parecen bugs; bugs de los que ya depende otro código.

Lo que engaña: vicios ocultos que el agente toma como verdad



Código muerto que parece vivo

Funciones sin callers, flags que nadie prende. El agente las copia como patrón.



Docs y comentarios viejos

El README describe la arquitectura de hace tres años. El agente le cree.



Tres formas de hacer lo mismo

Dos clientes HTTP, tres loggers, una migración a medias. ¿Cuál es la convención?



Tests que no testean

Flaky ignorados, mocks de todo, asserts vacíos. Verde no significa correcto.



Comportamiento fuera del código

Env vars, feature flags, código generado, inyección de dependencias. Grep no lo encuentra.



Dependencias fuera del repo

Otros servicios, jobs y scripts de otro equipo consumen tu salida. Desde acá no se ven.

Lo que falta, el agente lo **inventa**. Lo que engaña, lo **replica**.

Open source vs. empresa: dónde está escrito

El mismo contexto oculto, con distinta cantidad escrita.

Proyecto open source

Vive de que gente desconocida contribuya, así que tiene que escribirlo.

- **README** y **CONTRIBUTING** con build, tests y estilo
- CI pública que corre en cada PR
- el porqué, en issues y PRs abiertos
- linters que imponen las convenciones
- commits descriptivos: la historia se lee

Ej.: fzf declara `make test`, `make lint` y el `go:generate`.

Proyecto de empresa

Lo mantiene un equipo que ya sabe, así que casi nada se escribe.

- conocimiento tribal: "preguntale a Juan"
- decisiones en Slack, reuniones y tickets viejos
- setup local que anda en dos máquinas
- tests que solo corren en CI, con secretos
- commits "fix", "wip", "cambios"

Y el repo es uno de muchos: servicios, colas y bases de otros equipos.

En la empresa, **descubrir** también es preguntar a personas y leer tickets, no solo código. Y escribir un **AGENTS.md** es hacer para los agentes el **CONTRIBUTING.md** que nunca se hizo para los humanos.

Qué ve el agente: su contexto

El agente no "conoce" tu repo. Ve **solamente** lo que está en su context window, medida en tokens (~4 caracteres de código).

Una sesión típica, por dentro



Lo naranja entra **a medida que explora**: cada archivo leído y cada comando corrido ocupa lugar, sirva o no.

El repo no entra: tokens para leer *todo* el código fuente

ventana  400K de referencia

fzf  940 KB de Go → ~250K

tmux  2,9 MB de C → ~750K

Tokens $\approx$ bytes $\div$ 4 (unos 4 caracteres por token). Es una estimación, y el código suele dar más. Un repo de empresa es varias veces **tmux**.

Y aunque entrara

Más contexto no es mejor contexto. Con la ventana llena, lo importante compite con el ruido y el agente **atiende peor**. Al llegar al tope, **compacta**: resume y descarta detalle.

El contexto oculto del repo solo existe para el agente **si alguien lo pone en la ventana**.

El contexto en cada agente

Mismo problema en los tres. Cambian el tamaño, dónde se escriben las rules y qué pasa cuando se llena.

	Claude Code	Codex CLI	Grok Build
Ventana	200K - 1M, según modelo	~400K (1M según modelo)	256K (grok-build) a 1M (Grok 4.x)
Rules del repo	CLAUDE.md jerárquico + .claude/rules/	AGENTS.md jerárquico, tope 32 KiB : el resto se trunca sin aviso	AGENTS.md (también lee CLAUDE.md) + .grok/rules/, sin tope
Cómo lee el código	bajo demanda: grep, glob, read	bajo demanda: comandos de shell (rg, cat)	bajo demanda: read, list, search
Cuando se llena	auto-compacta; recarga CLAUDE.md y los últimos archivos	compacta en un resumen de traspaso	auto-compacta según la ventana configurada
Subagents	ventana propia, devuelven un resumen (Explore)	ventana propia, hasta 8 en paralelo	ventana propia, devuelven un resumen (explore)

Ninguno **indexa** tu repo de antemano: buscan y leen a medida que avanzan. **Lo que no buscan, no lo ven.**

Estrategias SDD: poner lo importante en la ventana

Cada tipo de contexto oculto tiene un lugar en disco donde se escribe, para que el agente lo lea **sin tener que redescubrirlo**.

 **Arquitectura existente**

→ **Notas de exploración**: el mapa con paths y funciones, no el código entero

 **Convenciones implícitas**

→ **CLAUDE.md / AGENTS.md**: cortas, porque se cargan en cada sesión

 **Deuda técnica**

→ **Invariantes y fuera de alcance** en la spec, cada uno con su chequeo

 **Documentación faltante**

→ **Context capsule**: las decisiones y el porqué, para la próxima spec

Y para cuidar la ventana misma:

Subagente de exploración – lee 25 archivos en su ventana y te devuelve la conclusión.

Sesión nueva por fase – la implementación arranca con la spec, no con el barro de explorar.

Superficie acotada – la spec dice qué leer y qué no. Menos lecturas, menos ruido.

CHECKPOINT

¿Lo limpiamos?

Le pediste al agente un cambio chico en `billing.py`. De paso encuentra `round(total - 0.001, 2)` y propone: "esto parece un bug, lo limpio". Nadie del equipo sabe por qué está así. **¿Qué hacés?**

A Aceptar: es obviamente un bug, y ya que está en el archivo... ❌
"De paso" es cómo se regresiona producción: agrandás la superficie de cambio sin spec ni revisión.

C Reescribir el módulo de facturación con un redondeo correcto. ❌
Big-bang sobre código que no entendés: diff enorme, imposible de revisar, regresiones por todos lados.

B **Frenar y descubrir:** buscar callers, tests y commits que expliquen esa línea. ✅
Deuda técnica: a veces un "bug" es algo de lo que otro código depende. Primero entendés, después decidís.

D Agregar un test que documente el comportamiento, y arreglarlo igual. ❌
El test está bien, pero arreglarlo sin saber quién depende de ese valor sigue siendo adivinar.

💡 En brownfield, adivinar es una regresión. **Una rareza puede estar sosteniendo el edificio.**

Descubrimiento antes de especificar

Mapeá el territorio antes de redibujarlo

Exploración vs. ejecución

Dos trabajos distintos. Mantenelos en **dos sesiones de agente distintas**.

Agente de exploración

Solo lectura. Busca, traza, resume. Su salida son **notas**, no ediciones.

Solo explorar, no editar. Mapeá el listado de objetos:
archivos, interfaces y riesgos.

Agente de implementación

Escribe código contra una **spec**, *después* del descubrimiento, con contexto limpio.

Implementá la Iteración 1 de esta spec. Tocá solo `lister.py`.

Cuán lejos puede ir la exploración, sin editar una línea

Código · ¿Quién llama a `calcular_total()`? Listá archivo:línea y cómo usa cada uno el valor que devuelve.

Historia · Con `git log -S` y `blame`: ¿qué commit agregó este `0.001`, y qué decía su PR?

Tests · ¿Qué tests cubren `billing.py`? ¿Cuáles están skipeados o son flaky? ¿Algo fija este valor?

Runtime · Corré la suite y el binario tal como están. Reportá qué pasa hoy: la línea de base.

Config y build · ¿Qué env vars, feature flags o código generado cambian este comportamiento?

Convenciones · ¿Cómo resolvieron este mismo problema otros tres módulos? ¿Hay un patrón, o son tres?

Fuera del repo · Buscá en issues, tickets y la wiki por qué existe este workaround. (vía `gh` o MCP)

Otros repos · ¿Qué servicios o scripts de la organización consumen este endpoint o este formato?

En paralelo · Lanzá tres subagents: flujo, historia y tests. Consolidá sus conclusiones en las notas.

¿Y si indexamos el repo?

Buscar en vivo gasta ventana. Se puede preparar el repo **antes**: cuatro estrategias, cada una con su costo.

Búsqueda agéntica

grep, glob y read en el momento.
Lo que hacen Claude Code,
Codex y Grok Build por defecto.

- ✓ siempre al día, exacta
- ✗ solo encuentra lo que se le ocurre buscar

Mapa estructural

tree-sitter o LSP: símbolos,
definiciones y referencias. Ej.: el
repo map de Aider, rankeado por
uso.

- ✓ barato, preciso
- ✗ sabe *dónde* está algo, no *por qué*

Índice semántico (RAG)

Embeddings de fragmentos y
búsqueda por similitud. Ej.: el
indexado de Cursor.

- ✓ busca por concepto
- ✗ trae lo *parecido*, no lo que depende; envejece con cada commit

Memoria curada

Lo que escribimos: notas de
exploración, rules, specs,
capsules.

- ✓ la única que tiene el *porqué*
- ✗ hay que escribirla y mantenerla

No compiten, se combinan: **el índice propone candidatos, la búsqueda agéntica los verifica** contra el código real, y **la memoria curada guarda** lo que se entendió.

Ningún índice reemplaza **saber qué preguntar**.

La salida: notas de exploración

El descubrimiento no está terminado hasta que está **escrito**. Las notas son el puente a la spec.

Módulos tocados

archivos y funciones donde va a vivir el cambio

Interfaces usadas

funciones, tipos y contratos existentes que vas a reusar

Riesgos identificados

callers que podrían romperse, convenciones, deuda cercana

Criterio de corte: terminaste de explorar cuando podés llenar estas tres cajas. Ni antes, ni mucho después.

Entrada · prompt al agente, en modo solo lectura

Solo explorar, sin editar nada. En fzf quiero agregar una acción **clear-history** que se bindee con `--bind`. Trazá cómo viaja una acción que ya existe, `prev-history`, desde el parseo hasta que se ejecuta. Devolveme notas con tres secciones: módulos tocados, interfaces usadas y riesgos. **Cada afirmación con `archivo:línea`.**



Salida · notas-exploracion.md · commit b1be3a8

Módulos tocados

- src/options.go:1909	switch que parsea las acciones
- src/terminal.go:557	type ActionType; dispatch en :7851
- src/history.go	clear() nuevo, junto a append()

Interfaces usadas

- History.append()	persiste con <code>os.WriteFile(..., 0600)</code>
- t.history != nil	guard: sin <code>--history</code> , es nil

Riesgos

- ActionType_string.go	GENERADO: regenerar, nunca editar
- append() al salir	reescribe el historial vaciado
- borrado irreversible	sin tecla por default

Paths y líneas **reales**, verificables abriendo el repo. El ejemplo completo está en [examples/02-brownfield/ejemplo-guiado/](#).

CHECKPOINT

¿Cómo seguís?

Pasaste 40 minutos explorando con el agente: leyó 25 archivos, siguió 3 pistas falsas, y ahora sabés qué módulos tocar y qué no romper. La spec ya está escrita. **¿Cómo arrancás la implementación?**

A En la **misma sesión**: ya tiene todo el contexto cargado. ❌
Tiene todo el barro cargado: los 25 archivos y las pistas falsas compiten con la spec por la ventana.

C Saltar la spec: ya entendiste el código, pedile que lo implemente directo. ❌
Entender no es acotar: sin spec no hay fuera de alcance, invariantes ni VCs contra los que verificar.

B **Sesión nueva**, sembrada con la spec y las notas de exploración. ✅
Exploración y ejecución separadas: el agente que escribe código recibe la conclusión destilada, no el camino.

D Sesión nueva, pegando la **transcripción completa** de la exploración. ❌
Es la opción A con otro nombre: el ruido viaja igual a la sesión nueva.



El descubrimiento ensucia el contexto. **Pasá la conclusión, no el camino.**

Acotar specs brownfield

Decí qué cambia – y qué *no* tiene que cambiar

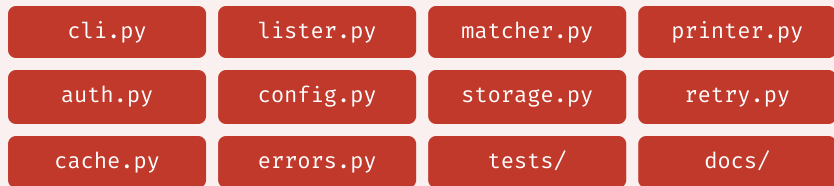
La superficie de cambio acotada

La parte **más chica** del sistema de la que tu spec es dueña. El mismo repo, dos pedidos:

Ancha y vaga

Mejorá el feature de búsqueda.

Lo que el agente puede tocar:



12

archivos en juego

0

invariantes

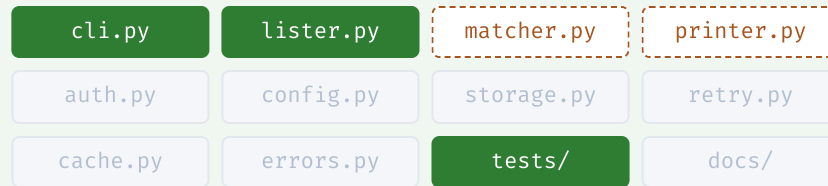
?

cuándo termina

Angosta y explícita

Agregá `--include=GLOB` en `cli.py` y `lister.py`, con su test. Dejá intactos `matcher.py` y `printer.py`.

Lo que el agente puede tocar:



3

piezas cambian

2

invariantes

1

sentada de revisión

■ cambia

□ intacto: invariante

□ fuera de alcance

■ sin límite

Una superficie ajustada es tu defensa contra el **drift**: el agente no deambula donde la spec no lo deja.

El fuera de alcance es parte del diseño

En greenfield listás qué construir. En brownfield también tenés que listar **qué no tocar**.

✅ Dentro de alcance

Los módulos que cambian, referenciados por path. El comportamiento nuevo, escrito como FRs + VCs.

❌ Fuera de alcance

Módulos que quedan intactos. Comportamiento que queda idéntico. Refactors postergados.

Agregá **invariantes** explícitos: "La firma pública de `search()` no cambia." "Los exit codes existentes se preservan."

Incremental antes que big-bang

Reescritura big-bang

un solo cambio
todo el módulo reescrito de una vez

Suite existente al final:



Diff imposible de revisar. Las regresiones aparecen todas juntas y no sabés qué parte del cambio rompió qué.

Cambio incremental



Suite existente después de cada spec:



Specs chicas que se revisan en minutos. Si algo se pone rojo, el culpable es el último incremento.

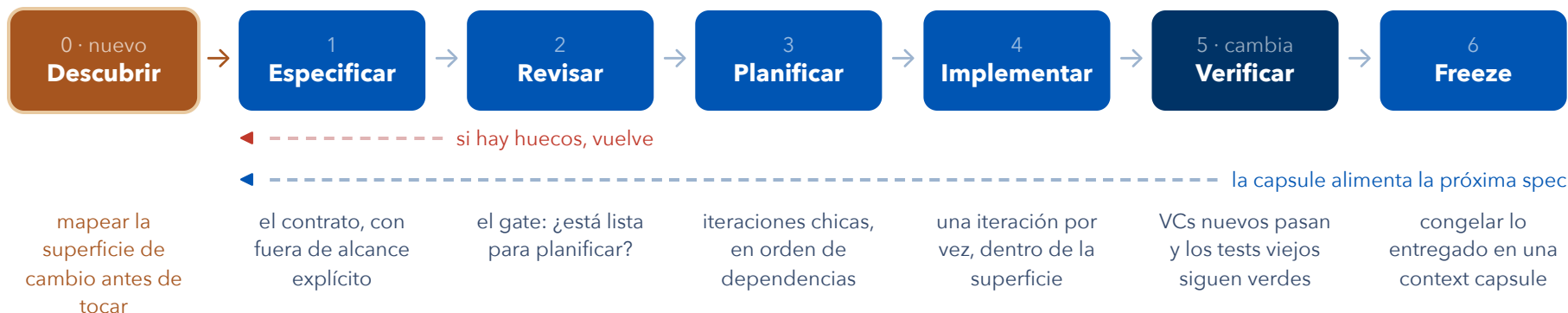
La suite de tests existente es tu red. Cada incremento tiene que dejarla **verde**.

Flujo SDD brownfield

Adaptar el ciclo a un codebase vivo

El pipeline, edición brownfield

El mismo pipeline de la Lección 1, con un paso adelante y un Verificar más exigente.



Qué cambia respecto de greenfield:

Descubrir – no se especifica contra un sistema que no mapeaste.

Verificar – que pasen tus VCs es necesario, pero no suficiente: la suite existente también tiene que quedar verde.

Freeze → Especificar – rara vez hay una sola spec: cada capsule es contexto de la siguiente.

Seguridad ante regresiones en Verificar

Brownfield le agrega una segunda pregunta a cada verificación.

¿Lo construí bien?

Cada FR/BR nuevo tiene un VC que pasa. La tabla de cobertura está completa.

Chequeá cada VC de la spec contra el código y mostrame la tabla de cobertura.

¿Rompí alguna otra cosa?

La suite de tests existente sigue verde. Los invariantes de la spec siguen valiendo.

Corré la suite existente completa y confirmá que la firma de search() y los exit codes no cambiaron.

Una iteración brownfield no termina cuando pasan tus VCs – termina cuando pasan tus VCs **y nada más se movió.**

CHECKPOINT

¿Terminó la iteración?

Agregaste `--include=GLOBAL` al lister de objetos. La spec tiene 4 VCs y los 4 pasan. El agente reporta: *"Iteración 1 completa"* . ¿Está terminada?

A Sí: los 4 VCs pasan, y los VCs son el contrato. 

Es la vara de greenfield. En brownfield es la mitad: responde "¿lo construí bien?", no "¿rompí algo?".

B No: falta correr la suite existente completa y chequear los invariantes. 

Termina cuando pasan tus VCs y nada más se movió: suite verde, firma de `search()` y exit codes intactos.

C No: falta congelarla con `freeze-spec` para darla por terminada. 

El freeze viene después de verificar y sirve a la próxima spec. No es lo que le falta a esta.

D No: hay que actualizar los tests viejos para que contemplen el flag nuevo. 

Si un test viejo falla, es una señal de regresión, no algo para "actualizar" hasta que pase.



Una iteración brownfield termina cuando pasan tus VCs **y nada más se movió.**

Context capsules con freeze-spec

Cuando una spec se entrega, **congelala** en un resumen compacto sobre el que construya la siguiente.

Propósito

qué entregó este cambio, en una oración

Interfaces y mapa

qué expone ahora, y dónde vive

Decisiones clave

qué se eligió, y qué se descartó

Tamaño, en líneas

notas		~190
spec		~150
capsule		~70

La próxima spec lee la **capsule**, no toda la historia.
Descubrimiento que no tenés que repetir.

Entrada · freeze-spec, con la spec ya entregada y verificada

Congelá esta spec en una context capsule: propósito, mapa, trampas, y decisiones con lo descartado. **Menos de la mitad de la spec.**



Salida · context-capsule.md · clear-history en fzf

Qué se construyó

Acción clear-history, bindeable: vacía el historial.

El mapa que costó descubrir

parseo	src/options.go (~1909)
constante	src/terminal.go (~557)
ejecución	src/terminal.go (~7850)

Agregar una acción toca esos lugares. Siempre.

La trampa

actiontype_string.go es generado: make generate.

Decisiones, y lo descartado

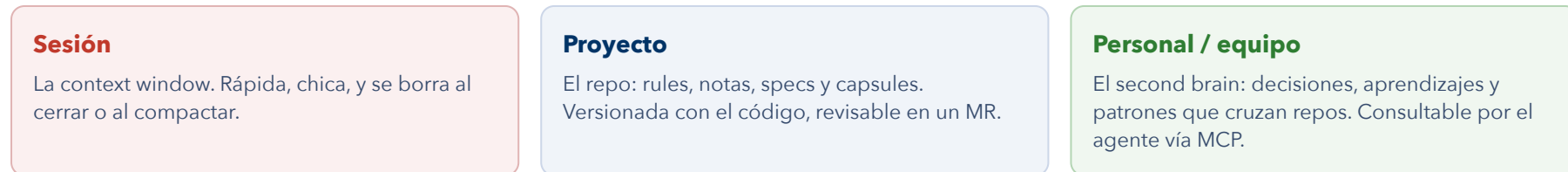
truncar el archivo	x borrarlo: pierde permisos
sin tecla por default	x CTRL-ALT-D: borrado accidental
no exponer en --listen	x más superficie de ataque

Second brain: memoria que sobrevive a la sesión

Un **second brain** es un sistema externo donde guardás lo que aprendés, para no tener que recordarlo. El agente lo necesita más que vos: **olvida todo al cerrar la sesión**.



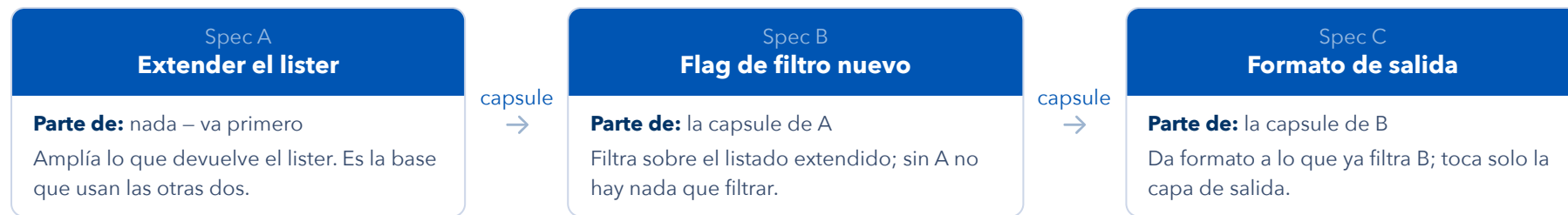
Tres capas de memoria



Cada sesión que **destila** algo hace más barata la siguiente.

Orden de dependencias entre specs

Trabajo brownfield grande = **varias specs acotadas**, hechas en el orden correcto.



Cada spec entrega, verifica en verde y congela una capsule que consume la siguiente. Ordená por **dependencia**, no por entusiasmo.

La secuencia le gana al tamaño: tres specs que quedan verdes le ganan a una que es demasiado grande para confiar.

¿Lo específico, o lo arreglo y ya?

No todo cambio se gana una spec completa. Que la ceremonia sea proporcional al riesgo.

Fix chico – andá directo

Typo, bug de una línea, cambio aislado con un test obvio. Radio de explosión bajo, fácil de revisar.

Arreglá el off-by-one en la numeración de líneas de `printer.py` y agregá un test de regresión.

Escribí una spec

Comportamiento nuevo, varios call sites, impacto entre módulos, o cualquier cosa con riesgo real de regresión.

No codees todavía – esto toca 3 call sites. Explorá primero, después escribí una spec del cambio.

La spec es una herramienta para manejar riesgo y ambigüedad – no un impuesto sobre cada commit.

CHECKPOINT

¿Spec o fix directo?

Cuatro cambios que llegan al mismo repo esta semana. ¿Cuáles se ganan una spec?

Un typo en el texto de `--help`.

Fix directo

Radio de explosión nulo y se revisa de un vistazo.

Renombrar un parámetro de `parseArgs()` que usan tres módulos.

Spec

Varios call sites: hay que descubrir quién lo usa y fijar invariantes.

Off-by-one en la numeración de líneas de `printer.py`, con un test que lo reproduce.

Fix directo

Aislado y con test obvio: arreglo + test de regresión.

Cambiar el formato de salida, que otros scripts del equipo parsean.

Spec

Consumidores que no ves desde el repo: riesgo real de regresión.



Ceremonia proporcional al riesgo: **radio de explosión x ambigüedad**.

Sin magia: ingeniería de contexto

Mismo agente, mismo modelo. Lo que cambia el resultado es **cómo le presentás el mundo**.

Prompt suelto

- redescubre el repo en cada sesión
- llena la ventana de lecturas que no sirven
- adivina las convenciones
- "terminé" lo decide el agente
- revisás todo el diff con desconfianza

Contexto diseñado

- arranca de la capsule y las notas
- explora con subagents; implementa con la spec
- lee las convenciones de rules cortas
- "terminé" lo deciden los VCs y los invariantes
- revisás contra la spec

En disco

no en el chat

Lo justo

no todo

Verificable

paths reales, chequeos

Por capas

sesión · proyecto · equipo

El modelo lo elegís una vez. **El contexto lo diseñás en cada tarea.**

Conceptos clave – repaso

-  **Descubrimiento** – leé, busca y trazá antes de especificar
-  **Exploración vs. ejecución** – separá entender de cambiar
-  **Superficie de cambio acotada** – la parte más chica de la que la spec es dueña
-  **Conciencia de regresión** – pasan los VCs nuevos y los tests viejos quedan verdes
-  **Context capsule** – resumen congelado que alimenta la próxima spec
-  **Ingeniería de contexto** – decidir qué ve el agente: en disco, lo justo, verificable, por capas

Briefing de la tarea

Análisis + spec brownfield – para hacer en casa, en equipo de trabajo

Tu tarea: `tmux` con SSH nativo

Agregá un **cliente SSH nativo** al codebase real de `tmux` – para que un pane abra una sesión remota **sin invocar** el binario `ssh`. **Solo Linux**.

El repo

Un proyecto C maduro de ~60k líneas: un event loop, un modelo de PTY/panes, una tabla de comandos, y una capa de portabilidad que vos no escribiste.

El cambio

Un comando nuevo que abre un pane con SSH de forma nativa (pensá `libssh`), en vez de correr `ssh` en una shell.

No lo van a implementar. El entregable es el **análisis y la spec** – el descubrimiento y el acotamiento de los que trata esta clase, sobre un codebase demasiado grande para tener en la cabeza.

"Solo Linux" no es un detalle – es el **límite de alcance** sobre el que gira toda la spec.

Cómo encarar tmux con SSH nativo

1 • Explorar (solo lectura, con agente)

Mapeá cómo un pane lanza un proceso hoy, y dónde engancharía un cliente nativo.

Solo explorar, sin editar: trazá cómo un pane nuevo lanza su proceso hijo – de la tabla de comandos al fork/exec.

¿Cómo aísla tmux el código específico de plataforma? Mostrame la capa compat/ y los chequeos de configure.

2 • Analizar y acotar

Nombrá la superficie de cambio acotada. Dibujá el límite "solo Linux" de forma explícita.

¿entrada en la tabla de comandos?

¿camino de spawn del pane?

¿libssh vs OpenSSH?

¿integración con el event loop?

¿auth (claves/agent)?

¿guarda de build para no-Linux?

Invariantes: los builds no-Linux siguen compilando (el feature se compila afuera); los comandos existentes y el modelo de PTY/panes no cambian.

Tarea (para antes de la Lección 3)

Trabajen en su **equipo de trabajo**. Lleven el `tmux` real de "repo desconocido" a una **spec acotada y revisable** de un cliente SSH nativo, solo para Linux.

Análisis / notas de exploración

Módulos tocados, interfaces reusadas (camino de spawn, tabla de comandos, event loop), riesgos y la historia de compat/build

Spec brownfield

Alcance dentro/fuera explícito, **límite solo-Linux**, invariantes, FRs + VCs, referenciando archivos reales

Sin implementación

El valor está en el descubrimiento y el alcance – una spec desde la que otro equipo podría construir

Corran en serio el frente del pipeline: **Descubrir → Especificar → Revisar**

Entrega y presentación



Mail a la cátedra con el **link al repo** del equipo y el **hash del commit**



Hasta el **jueves al mediodía** de la semana anterior a la Lección 3



Al inicio de la Lección 3 se **sortea un equipo** para presentar



Miramos: disciplina de alcance, límite solo-Linux, invariantes y cobertura de VCs

Vengan todos listos para mostrar: el sorteo es en el momento. La consigna completa está en `examples/02-brownfield/tarea/enunciado.md`.

¿Preguntas?

Próxima: Lección 3 – Escribir skills y agentes para SDD

Encodear el playbook del equipo para que toda sesión de agente arranque con él.